

## Objetivo

O objetivo desta atividade prática em laboratório é implementar uma **árvore binária de busca** utilizando os conceitos de programação orientada a objetos. A implementação deve explorar a propriedade fundamental dessa estrutura: para cada nó, os valores da subárvore à esquerda são menores que o valor armazenado no nó, enquanto os valores da subárvore à direita são maiores ou iguais.

Além disso, a estrutura implementada deverá ser utilizada em uma aplicação prática simples: o armazenamento e a consulta de **códigos numéricos de estudantes**.

## Parte 1 – Implementação da Árvore Binária de Busca

1. Inicialmente crie os seguintes arquivos fonte:

- **NoArvoreBusca**: classe que representa um nó da árvore;
- **ArvoreBinariaBusca**: classe que representa a árvore binária de busca;
- **ArvoreBuscaMain**: classe com o método principal para demonstrar o funcionamento da árvore e da aplicação prática.

2. Estrutura sugerida para a classe **NoArvoreBusca**:

```
public class NoArvoreBusca {  
    private int info;  
    private NoArvoreBusca esq;  
    private NoArvoreBusca dir;  
  
    public NoArvoreBusca(int info) {  
        this.info = info;  
        this.esq = null;  
        this.dir = null;  
    }  
  
    // getters e setters  
}
```

3. Significado dos atributos:

- **info**: valor inteiro armazenado no nó;
- **esq**: referência para a subárvore esquerda;
- **dir**: referência para a subárvore direita.

4. A classe `ArvoreBinariaBusca` deve possuir, no mínimo, um atributo para armazenar a referência à raiz da árvore:

```
private NoArvoreBusca raiz;
```

5. Métodos a serem implementados na classe `ArvoreBinariaBusca`:

- (a) `public ArvoreBinariaBusca()`: construtor que instancia uma árvore vazia;
- (b) `public boolean vazia()`: retorna `true` se a árvore estiver vazia, ou `false` caso contrário;
- (c) `public NoArvoreBusca busca(int v)`: busca o valor `v` na árvore e retorna o nó encontrado, ou `null` caso o valor não esteja na árvore;
- (d) `private NoArvoreBusca busca(NoArvoreBusca no, int v)`: método auxiliar recursivo para busca;
- (e) `public void insere(int v)`: insere o valor `v` na árvore, respeitando a propriedade da árvore binária de busca;
- (f) `private NoArvoreBusca insere(NoArvoreBusca no, int v)`: método auxiliar recursivo para inserção, retornando a eventual nova raiz da subárvore;
- (g) `public void retira(int v)`: remove o valor `v` da árvore, se ele existir;
- (h) `private NoArvoreBusca retira(NoArvoreBusca no, int v)`: método auxiliar recursivo para remoção, retornando a eventual nova raiz da subárvore;
- (i) `public boolean pertence(int v)`: retorna `true` se o valor `v` pertence à árvore, ou `false` caso contrário;
- (j) `public int numNos()`: retorna a quantidade total de nós da árvore;
- (k) `public int folhas()`: retorna a quantidade de nós do tipo folha;
- (l) `public int altura()`: retorna a altura da árvore;
- (m) `public String toString()`: retorna os valores da árvore em ordem crescente;
- (n) `public String toStringDecrescente()`: retorna os valores da árvore em ordem decrescente.

6. A operação de busca deve explorar a propriedade de ordenação da árvore:

- se a árvore estiver vazia, o valor não foi encontrado;
- se `v` for menor que o valor do nó atual, a busca continua na subárvore esquerda;
- se `v` for maior que o valor do nó atual, a busca continua na subárvore direita;
- caso contrário, o valor foi encontrado.

7. A operação de inserção deve retornar, em seu método auxiliar recursivo, a eventual nova raiz da subárvore. Isso é necessário porque uma inserção pode criar a raiz de uma subárvore que antes era vazia.

8. A operação de remoção deve tratar os três casos vistos em sala:

- remoção de nó folha;
- remoção de nó com apenas um filho;
- remoção de nó com dois filhos.

9. No caso de remoção de um nó com dois filhos, utilize uma das estratégias abaixo:

- substituir o valor do nó pelo maior valor da subárvore esquerda;
- substituir o valor do nó pelo menor valor da subárvore direita.

Escolha apenas uma das estratégias e mantenha-a de forma consistente.

10. A impressão em ordem crescente deve percorrer a árvore em ordem simétrica:

`subarvore esquerda, raiz, subarvore direita`

11. A impressão em ordem decrescente deve percorrer a árvore na ordem inversa:

`subarvore direita, raiz, subarvore esquerda`

## Parte 2 – Problema Aplicado: Consulta de Códigos de Estudantes

Após implementar a árvore binária de busca, utilize-a para representar um conjunto de **códigos numéricos de estudantes**. Cada código será armazenado em um nó da árvore.

Implemente, na classe `ArvoreBuscaMain`, as seguintes funcionalidades:

1. Criar uma árvore binária de busca vazia;
2. Inserir pelo menos 15 códigos inteiros na árvore;
3. Imprimir os códigos em ordem crescente;
4. Imprimir os códigos em ordem decrescente;
5. Buscar pelo menos três códigos existentes;
6. Buscar pelo menos três códigos inexistentes;
7. Informar o número total de nós da árvore;
8. Informar a quantidade de folhas;
9. Informar a altura da árvore;
10. Remover um nó folha;
11. Remover um nó com apenas um filho;
12. Remover um nó com dois filhos;
13. Após cada remoção, imprimir novamente a árvore em ordem crescente para verificar se a propriedade da árvore binária de busca foi preservada.

**Sugestão de dados para teste:**

50, 30, 70, 20, 40, 60, 80, 35, 45, 55, 65, 75, 85, 10, 25

Com essa sequência, a árvore terá valores distribuídos nos dois lados da raiz, facilitando os testes de busca, impressão e remoção.

## Questões para Análise

Após executar os testes, responda no próprio código, em forma de comentários, ou em um pequeno arquivo de texto:

1. Por que a impressão em ordem simétrica mostra os valores em ordem crescente?
2. Por que a busca em uma árvore binária de busca não precisa visitar todos os nós, em geral?
3. O que pode acontecer com o desempenho da busca se a árvore ficar muito desbalanceada?
4. Qual foi o caso de remoção mais trabalhoso de implementar? Explique.

## Observações

- A árvore deve armazenar valores inteiros;
- Valores repetidos devem ser inseridos na subárvore direita;
- Utilize adequadamente os princípios de encapsulamento da programação orientada a objetos;
- Sempre que necessário, implemente métodos privados auxiliares recursivos;
- Tenha cuidado para atualizar corretamente as referências das subárvores durante inserções e remoções;
- Implemente na classe **ArvoreBuscaMain** um exemplo completo demonstrando o funcionamento da árvore e da aplicação prática.

## Observação Final

Ao final da atividade, o programa deve demonstrar claramente que a árvore binária de busca mantém os dados organizados de forma a permitir buscas, inserções, remoções e impressões ordenadas usando a propriedade de ordenação da estrutura.